Hill Climbing for 4 Queens

Code:

```
import random

def cost(state):

    conflicts = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):

            if state[i] == state[j]:
                conflicts += 1

            elif abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1
    return conflicts

def hill_climbing(initial_state=None):

    if initial_state is None:
        current = [random.randint(0, 3) for _ in range(4)]
    else:
        current = list(initial_state)

    print("Initial state:", current)
    print("Initial cost:", cost(current))

    while True:
        best_neighbor = current[:]
        best_cost = cost(current)


        for col in range(4):
            for row in range(4):
                if row != current[col]:
                    neighbor = current[:]
                    neighbor[col] = row
                    neighbor_cost = cost(neighbor)


                    if neighbor_cost < best_cost:
                        best_neighbor = neighbor
                        best_cost = neighbor_cost


        if best_cost >= cost(current):
            print("No better neighbor found.")
            break
```

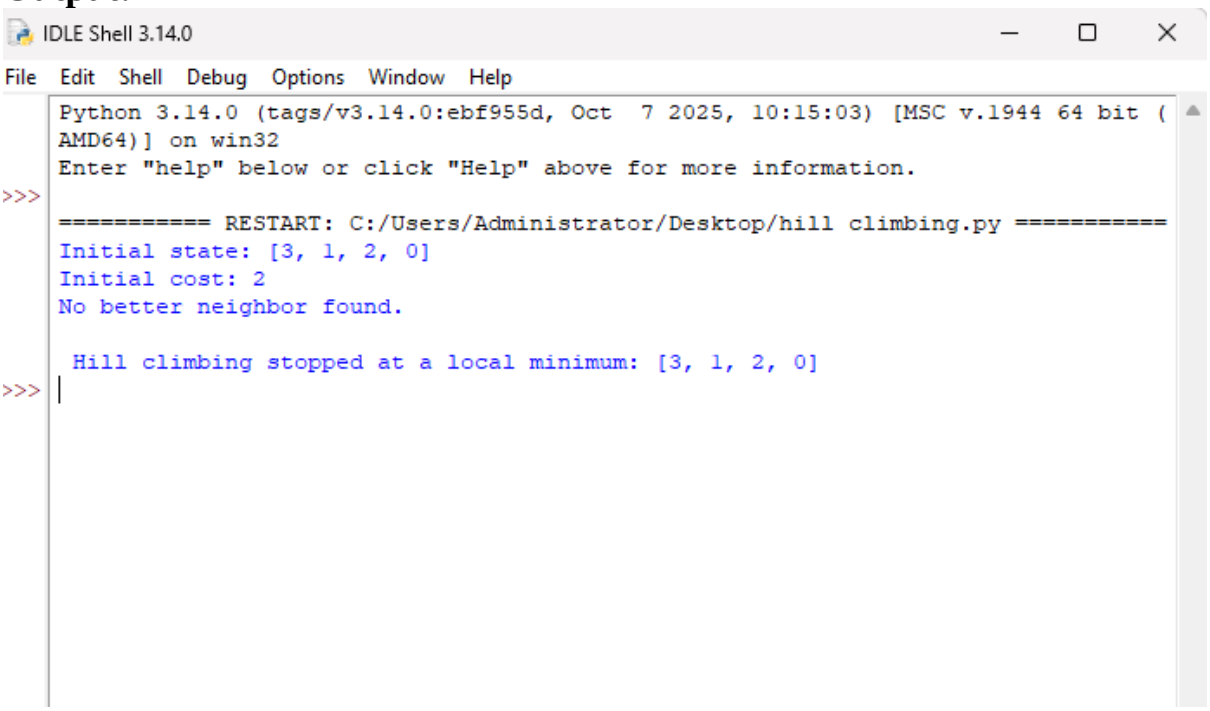
```
        current = best_neighbor
        print("Moved to state:", current, "with cost:", cost(current))

        if cost(current) == 0:
            break

    if cost(current) == 0:
        print("\n Solution found:", current)
    else:
        print("\n Hill climbing stopped at a local minimum:", current)

hill_climbing([3, 1, 2, 0])
```

Output:



Simulated Annealing for 8-Queens

Code:

```
import random
import math

def cost(state):
    conflicts = 0
    n = len(state)

    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j]:
                conflicts += 1
            elif abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1

    return conflicts

def simulated_annealing():
    n = 8

    current = [random.randint(0, n - 1) for _ in range(n)]

    temperature = 100.0
    cooling_rate = 0.995
    minimum_temperature = 0.001

    print("Initial state:", current)
    print("Initial cost:", cost(current))

    while temperature > minimum_temperature and cost(current) != 0:
        next_state = current[:]
        col = random.randint(0, n - 1)
        new_row = random.randint(0, n - 1)

        while new_row == next_state[col]:
            new_row = random.randint(0, n - 1)

        next_state[col] = new_row

        current_cost = cost(current)
        next_cost = cost(next_state)

        delta_e = current_cost - next_cost

        if delta_e > 0:
            current = next_state
        else:
            probability = math.exp(delta_e / temperature)
```

```
            if random.random() < probability:
                current = next_state

        temperature *= cooling_rate

    print("\nFinal state:", current)
    print("Final cost:", cost(current))

    if cost(current) == 0:
        print("Solution found!")
    else:
        print("No solution found within the cooling schedule.")

simulated_annealing()
```

Output:

